

Go-SyncFlow - API 接口文档

企业级身份同步与管理解决方案

版本: v3.5

更新日期: 2026-02-25

文档状态: 正式发布

Go-SyncFlow Team

企业级统一身份管理 • 多源数据同步 • SSO 单点登录 • VPN 集成

目录

1	Go-SyncFlow - API 接口文档	5
1.1	1. 认证方式	5
1.1.1	1.1.1 AppID + AppKey 直接认证	5
1.2	2. 调用示例	6
1.2.1	2.1 cURL / Shell	6
1.2.2	2.2 Python	6
1.2.3	2.3 JavaScript / Node.js	6
1.2.4	2.4 Go	7
1.3	3. 响应格式	7
1.3.1	3.1 成功响应	7
1.3.2	3.2 分页响应	8
1.3.3	3.3 错误响应	8
1.4	4. 用户管理接口	8
1.4.1	4.1 获取用户列表	8
1.4.2	4.2 获取用户详情	9
1.4.3	4.3 创建用户	9
1.4.4	4.4 更新用户	10
1.4.5	4.5 删除用户	10
1.4.6	4.6 更新用户状态	10
1.4.7	4.7 重置用户密码	10
1.5	5. 用户组接口	10
1.5.1	5.1 获取用户组列表	10
1.5.2	5.2 创建用户组	11
1.5.3	5.3 更新用户组	11
1.5.4	5.4 删除用户组	11
1.6	6. 角色接口	11
1.6.1	6.1 获取角色列表	11
1.6.2	6.2 获取角色详情	11
1.6.3	6.3 获取角色权限	11
1.7	7. 单点登录 (SSO) 接口	12
1.7.1	7.1 获取 SSO 应用列表	12
1.7.2	7.2 创建 SSO 应用	12
1.7.3	7.3 获取 SSO 应用详情	12
1.7.4	7.4 更新 SSO 应用	13
1.7.5	7.5 删除 SSO 应用	13
1.7.6	7.6 获取 SSO 协议配置	13
1.7.7	7.7 更新 SSO 协议配置	13

1.7.8	7.8 获取 SSO 属性映射	14
1.7.9	7.9 更新 SSO 属性映射	14
1.7.10	7.10 获取 SSO 授权范围	14
1.7.11	7.11 更新 SSO 授权范围	14
1.7.12	7.12 获取 SSO 审计日志	14
1.8	8. VPN 管理接口	15
1.8.1	8.1 获取 VPN 服务状态	15
1.8.2	8.2 启动 VPN 服务	15
1.8.3	8.3 停止 VPN 服务	15
1.8.4	8.4 获取 VPN 配置	15
1.8.5	8.5 更新 VPN 配置	16
1.8.6	8.6 获取 VPN 用户组列表	16
1.8.7	8.7 创建 VPN 用户组	16
1.8.8	8.8 更新 VPN 用户组	17
1.8.9	8.9 删除 VPN 用户组	17
1.8.10	8.10 获取 VPN 在线用户	17
1.8.11	8.11 踢出 VPN 用户	17
1.8.12	8.12 获取 VPN 日志	17
1.9	9. 连接器管理接口	18
1.9.1	9.1 获取上游连接器列表	18
1.9.2	9.2 创建上游连接器	18
1.9.3	9.3 测试连接器	19
1.9.4	9.4 获取下游连接器列表	19
1.9.5	9.5 创建下游连接器	19
1.9.6	9.6 获取连接器类型	20
1.10	10. 同步规则接口	21
1.10.1	10.1 获取上游同步规则列表	21
1.10.2	10.2 创建上游同步规则	21
1.10.3	10.3 触发同步	21
1.10.4	10.4 获取字段映射	21
1.10.5	10.5 更新字段映射	22
1.11	11. 通知渠道接口	22
1.11.1	11.1 获取通知渠道列表	22
1.11.2	11.2 创建通知渠道	23
1.11.3	11.3 测试通知渠道	24
1.11.4	11.4 更新通知渠道	25
1.11.5	11.5 删除通知渠道	25
1.12	12. HTTPS 证书接口	25
1.12.1	12.1 获取 HTTPS 配置	25
1.12.2	12.2 更新 HTTPS 配置	25
1.12.3	12.3 上传 SSL 证书	26
1.12.4	12.4 删除 SSL 证书	26
1.13	13. 安全中心接口	26

1.13.1	13.1 获取安全仪表盘	26
1.13.2	13.2 获取安全事件	26
1.13.3	13.3 获取 IP 黑名单	27
1.13.4	13.4 添加 IP 黑名单	27
1.13.5	13.5 获取 IP 白名单	27
1.13.6	13.6 添加 IP 白名单	27
1.13.7	13.7 获取活跃会话	27
1.13.8	13.8 终止会话	27
1.13.9	13.9 解锁账户	27
1.14	14. 日志查询接口	28
1.14.1	14.1 登录日志 (Open API)	28
1.14.2	14.2 操作日志 (Open API)	28
1.14.3	14.3 同步日志 (Open API)	28
1.14.4	14.4 API 调用日志	28
1.14.5	14.5 VPN 日志	29
1.15	15. 系统状态接口	29
1.15.1	15.1 获取系统状态 (Open API)	29
1.16	16. 触发同步接口	30
1.16.1	16.1 触发钉钉同步 (Open API)	30
1.16.2	16.2 获取同步状态 (Open API)	30
1.16.3	16.3 触发全量同步	30
1.17	17. API 密钥管理接口	30
1.17.1	17.1 获取 API 密钥列表	30
1.17.2	17.2 创建 API 密钥	30
1.17.3	17.3 重置 API 密钥	31
1.17.4	17.4 启用/禁用 API 密钥	31
1.17.5	17.5 删除 API 密钥	31
1.18	18. 密码代理认证接口	31
1.18.1	18.1 获取密码代理配置	31
1.18.2	18.2 更新密码代理配置	32
1.18.3	18.3 测试密码代理	32
1.18.4	18.4 用户认证接口 (OpenAPI 专用)	32
1.18.5	18.5 用户同步接口 (含密码哈希)	34
1.19	19. API 安全限制	36
1.19.1	19.1 禁止的操作	36
1.19.2	19.2 日志清理机制	36
1.20	20. 接口汇总表	37
1.20.1	20.1 认证接口	37
1.20.2	20.2 用户管理 (不含密码)	37
1.20.3	20.3 用户同步 (含密码哈希)	38
1.20.4	20.4 单点登录 (SSO)	38
1.20.5	20.5 VPN 管理	39
1.20.6	20.6 上游同步	40

1.20.7	下游同步	40
1.20.8	日志管理	41
1.20.9	通知渠道	41
1.20.10	消息模板与策略	42
1.20.11	安全中心	42
1.20.12	系统设置	43
1.20.13	API 密钥管理	43
1.20.14	密码代理认证	44
1.21	20. 错误码参考	44
1.22	21. 安全说明	45
1.22.1	API 安全限制	45
1.22.2	频率限制	45

1 Go-SyncFlow - API 接口文档

版本：v3.7 / 更新日期：2026-02-25

本文档面向开发者，描述 Go-SyncFlow 所有 API 接口规范与调用示例。

1.1 1. 认证方式

1.1.1 AppID + AppKey 直接认证

所有 Open API 接口通过 `/api/open/*` 路径访问，使用 **AppID + AppKey 直接认证**。

Open API 支持的功能与管理后台完全一致，包括：用户管理、SSO 应用、VPN、连接器、同步、通知、安全等所有功能。

获取凭证

1. 登录管理后台 → **系统设置** → **API 密钥**
2. 点击「新建密钥」，设置名称和权限
3. 保存后获取 AppID 和 AppKey

注意：*AppKey* 只在创建时显示一次，请妥善保管。

请求头

Header	类型	必填	说明
X-App-ID	string	是	AppID
X-App-Key	string	是	AppKey（原始密钥）
Content-Type	string	POST/PUT	application/json

安全说明：系统仅支持请求头认证，不支持 *URL* 参数传递 *AppKey*，避免密钥出现在 *URL*、日志和浏览器历史中。

1.2 2. 调用示例

1.2.1 2.1 cURL / Shell

```
#!/bin/bash
APP_ID="your_app_id"
APP_KEY="your_app_key"
BASE_URL="http://localhost:8080"

curl -X GET "${BASE_URL}/api/open/users?page=1&size=10" \
  -H "X-App-ID: ${APP_ID}" \
  -H "X-App-Key: ${APP_KEY}"
```

1.2.2 2.2 Python

```
import requests

APP_ID = "your_app_id"
APP_KEY = "your_app_key"
BASE_URL = "http://localhost:8080"

def get_headers():
    return {
        "X-App-ID": APP_ID,
        "X-App-Key": APP_KEY,
    }

response = requests.get(
    f"{BASE_URL}/api/open/users",
    params={"page": 1, "size": 10},
    headers=get_headers()
)
print(response.json())
```

1.2.3 2.3 JavaScript / Node.js

```
const axios = require("axios");

const APP_ID = "your_app_id";
const APP_KEY = "your_app_key";
const BASE_URL = "http://localhost:8080";

function getHeaders() {
    return {
        "X-App-ID": APP_ID,
        "X-App-Key": APP_KEY
    }
}
```

```
};  
}  
  
axios.get(BASE_URL + "/api/open/users", {  
  params: { page: 1, size: 10 },  
  headers: getHeaders()  
}).then(res => console.log(res.data));
```

1.2.4 2.4 Go

```
package main  
  
import (  
    "fmt"  
    "io"  
    "net/http"  
)  
  
func main() {  
    appID := "your_app_id"  
    appKey := "your_app_key"  
    baseURL := "http://localhost:8080"  
  
    req, _ := http.NewRequest("GET", baseURL+"/api/open/users", nil)  
    req.Header.Set("X-App-ID", appID)  
    req.Header.Set("X-App-Key", appKey)  
  
    resp, _ := http.DefaultClient.Do(req)  
    body, _ := io.ReadAll(resp.Body)  
    fmt.Println(string(body))  
}
```

1.3 3. 响应格式

1.3.1 成功响应

```
{  
  "success": true,  
  "message": "操作成功",  
  "data": { ... }  
}
```

1.3.2 分页响应

```
{
  "success": true,
  "data": {
    "list": [ ... ],
    "total": 100,
    "page": 1,
    "size": 20
  }
}
```

1.3.3 错误响应

```
{
  "success": false,
  "message": "错误原因",
  "code": "ERROR_CODE"
}
```

1.4 4. 用户管理接口

1.4.1 4.1 获取用户列表

```
GET /api/open/users
```

参数	类型	说明
page	int	页码, 默认 1
size	int	每页数量, 默认 20
keyword	string	搜索关键词
status	string	active / disabled
roleId	int	角色 ID
groupId	int	用户组 ID

响应

```
{
  "success": true,
```

```
"data": {
  "list": [
    {
      "id": 1,
      "username": "zhangsan",
      "displayName": "张三",
      "email": "zhangsan@example.com",
      "phone": "13800138000",
      "status": "active",
      "roles": [{"id": 2, "name": "普通用户"}],
      "groups": [{"id": 1, "name": "研发部"}],
      "createdAt": "2026-01-01T10:00:00Z"
    }
  ],
  "total": 100,
  "page": 1,
  "size": 10
}
```

安全说明：此接口**不返回用户密码**。密码字段 (*password_hash*、*samba_nt_password*) 在响应中被自动隐藏，确保用户管理操作的安全性。如需获取密码哈希用于下游系统同步，请使用专用的[用户同步接口](#)。

1.4.2 4.2 获取用户详情

```
GET /api/open/users/:id
```

1.4.3 4.3 创建用户

```
POST /api/open/users
```

请求体

```
{
  "username": "lisi",
  "displayName": "李四",
  "email": "lisi@example.com",
  "phone": "13900139000",
  "password": "optional",
  "roleIds": [2],
  "groupIds": [1],
  "status": "active"
}
```

1.4.4 4.4 更新用户

```
PUT /api/open/users/:id
```

1.4.5 4.5 删除用户

```
DELETE /api/open/users/:id
```

1.4.6 4.6 更新用户状态

```
PUT /api/open/users/:id/status
```

请求体

```
{"status": "disabled"}
```

1.4.7 4.7 重置用户密码

```
PUT /api/open/users/:id/reset-password
```

1.5 5. 用户组接口

1.5.1 5.1 获取用户组列表

```
GET /api/open/groups
```

响应

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "研发部",
      "parentId": 0,
      "userCount": 50,
      "children": [
        {"id": 2, "name": "前端组", "parentId": 1, "userCount": 20}
      ]
    }
  ]
}
```

```
]
}
```

1.5.2 5.2 创建用户组

```
POST /api/open/groups
```

1.5.3 5.3 更新用户组

```
PUT /api/open/groups/:id
```

1.5.4 5.4 删除用户组

```
DELETE /api/open/groups/:id
```

1.6 6. 角色接口

1.6.1 6.1 获取角色列表

```
GET /api/open/roles
```

1.6.2 6.2 获取角色详情

```
GET /api/open/roles/:id
```

1.6.3 6.3 获取角色权限

```
GET /api/open/roles/:id/permissions
```

响应

```
{
  "success": true,
  "data": {
    "permissions": ["user:list", "user:create", "log:login"]
  }
}
```

1.7 7. 单点登录 (SSO) 接口

1.7.1 7.1 获取 SSO 应用列表

```
GET /api/open/sso/apps
```

响应

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "OA系统",
      "code": "oa",
      "type": "cas",
      "status": "active",
      "icon": "https://...",
      "loginUrl": "https://oa.example.com",
      "createdAt": "2026-01-01T00:00:00Z"
    }
  ]
}
```

1.7.2 7.2 创建 SSO 应用

```
POST /api/open/sso/apps
```

请求体

```
{
  "name": "HR系统",
  "code": "hr",
  "type": "oauth2",
  "icon": "https://...",
  "description": "人力资源管理系统",
  "loginUrl": "https://hr.example.com"
}
```

1.7.3 7.3 获取 SSO 应用详情

```
GET /api/open/sso/apps/:id
```

1.7.4 7.4 更新 SSO 应用

```
PUT /api/open/sso/apps/:id
```

1.7.5 7.5 删除 SSO 应用

```
DELETE /api/open/sso/apps/:id
```

1.7.6 7.6 获取 SSO 协议配置

```
GET /api/open/sso/apps/:id/protocols
```

响应

```
{
  "success": true,
  "data": {
    "cas": {
      "enabled": true,
      "serviceUrl": "https://app.example.com/*"
    },
    "oauth2": {
      "enabled": true,
      "clientId": "xxx",
      "clientSecret": "xxx",
      "redirectUri": ["https://app.example.com/callback"],
      "grantTypes": ["authorization_code", "refresh_token"]
    },
    "saml": {
      "enabled": false
    },
    "oidc": {
      "enabled": true,
      "clientId": "xxx"
    }
  }
}
```

1.7.7 7.7 更新 SSO 协议配置

```
PUT /api/open/sso/apps/:id/protocols
```

1.7.8 7.8 获取 SSO 属性映射

```
GET /api/open/sso/apps/:id/mappings
```

1.7.9 7.9 更新 SSO 属性映射

```
PUT /api/open/sso/apps/:id/mappings
```

1.7.10 7.10 获取 SSO 授权范围

```
GET /api/open/sso/apps/:id/grants
```

1.7.11 7.11 更新 SSO 授权范围

```
PUT /api/open/sso/apps/:id/grants
```

请求体

```
{
  "grantType": "all",
  "roleIds": [],
  "groupIds": [],
  "userIds": []
}
```

1.7.12 7.12 获取 SSO 审计日志

```
GET /api/open/sso/logs
```

参数	类型	说明
appId	int	应用 ID
username	string	用户名
action	string	login/logout
startDate	string	开始日期
endDate	string	结束日期

1.8 8. VPN 管理接口

1.8.1 8.1 获取 VPN 服务状态

```
GET /api/open/vpn/status
```

响应

```
{
  "success": true,
  "data": {
    "running": true,
    "serverAddr": ":443",
    "linkMode": "tun",
    "ipRange": "192.168.90.0/24",
    "onlineUsers": 15,
    "totalConnections": 1250
  }
}
```

1.8.2 8.2 启动 VPN 服务

```
POST /api/open/vpn/start
```

1.8.3 8.3 停止 VPN 服务

```
POST /api/open/vpn/stop
```

1.8.4 8.4 获取 VPN 配置

```
GET /api/open/vpn/config
```

响应

```
{
  "success": true,
  "data": {
    "serverAddr": ":443",
    "linkMode": "tun",
    "ipRange": "192.168.90.0/24",
    "dns": ["8.8.8.8", "114.114.114.114"],
    "mtu": 1400,
    "maxClients": 100,
    "sessionTimeout": 86400
  }
}
```

```
}  
}
```

1.8.5 8.5 更新 VPN 配置

```
PUT /api/open/vpn/config
```

1.8.6 8.6 获取 VPN 用户组列表

```
GET /api/open/vpn/groups
```

响应

```
{  
  "success": true,  
  "data": [  
    {  
      "id": 1,  
      "name": "默认组",  
      "allowedRoutes": ["10.0.0.0/8", "192.168.0.0/16"],  
      "bandwidth": 0,  
      "maxSessions": 5,  
      "userCount": 100  
    }  
  ]  
}
```

1.8.7 8.7 创建 VPN 用户组

```
POST /api/open/vpn/groups
```

请求体

```
{  
  "name": "研发组",  
  "allowedRoutes": ["10.0.0.0/8"],  
  "bandwidth": 10240,  
  "maxSessions": 3,  
  "authType": "local",  
  "enable2FA": true  
}
```

1.8.8 8.8 更新 VPN 用户组

```
PUT /api/open/vpn/groups/:id
```

1.8.9 8.9 删除 VPN 用户组

```
DELETE /api/open/vpn/groups/:id
```

1.8.10 8.10 获取 VPN 在线用户

```
GET /api/open/vpn/online
```

响应

```
{
  "success": true,
  "data": [
    {
      "username": "zhangsan",
      "clientIp": "192.168.90.10",
      "publicIp": "123.45.67.89",
      "connectedAt": "2026-02-22T10:00:00Z",
      "bytesIn": 1024000,
      "bytesOut": 2048000
    }
  ]
}
```

1.8.11 8.11 踢出 VPN 用户

```
POST /api/open/vpn/kick
```

请求体

```
{"username": "zhangsan"}
```

1.8.12 8.12 获取 VPN 日志

```
GET /api/open/vpn/logs
```

参数	类型	说明
username	string	用户名
action	string	connect/disconnect
startDate	string	开始日期
endDate	string	结束日期

1.9 9. 连接器管理接口

1.9.1 9.1 获取上游连接器列表

```
GET /api/open/sync/upstream/connectors
```

响应

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "钉钉主账号",
      "type": "im_dingtalk",
      "typeName": "钉钉 DingTalk",
      "status": "active",
      "lastSyncAt": "2026-02-22T02:00:00Z"
    }
  ]
}
```

1.9.2 9.2 创建上游连接器

```
POST /api/open/sync/upstream/connectors
```

请求体（钉钉示例）

```
{
  "name": "钉钉主账号",
  "type": "im_dingtalk",
  "config": {
    "corpId": "xxx",
    "appKey": "xxx",
  }
}
```

```
    "appSecret": "xxx"
  }
}
```

请求体 (LDAP 示例)

```
{
  "name": "AD服务器",
  "type": "ldap_ad",
  "config": {
    "host": "ldap.example.com",
    "port": 389,
    "bindDN": "cn=admin,dc=example,dc=com",
    "bindPassword": "xxx",
    "baseDN": "dc=example,dc=com",
    "userFilter": "(objectClass=person)",
    "useTLS": true
  }
}
```

1.9.3 9.3 测试连接器

```
POST /api/open/sync/upstream/connectors/:id/test
```

响应

```
{
  "success": true,
  "data": {
    "connected": true,
    "userCount": 1500,
    "departmentCount": 50,
    "message": "连接成功"
  }
}
```

1.9.4 9.4 获取下游连接器列表

```
GET /api/open/sync/downstream/connectors
```

1.9.5 9.5 创建下游连接器

```
POST /api/open/sync/downstream/connectors
```

请求体 (AD 示例)

```
{
  "name": "AD目录服务",
  "type": "ldap_ad",
  "config": {
    "host": "ad.example.com",
    "port": 636,
    "bindDN": "cn=admin,dc=example,dc=com",
    "bindPassword": "xxx",
    "baseDN": "ou=users,dc=example,dc=com",
    "useTLS": true
  }
}
```

1.9.6 9.6 获取连接器类型

```
GET /api/open/sync/connector-types
```

响应

```
{
  "success": true,
  "data": [
    {"type": "im_dingtalk", "label": "钉钉 DingTalk", "category": "im", "upstream": true, "downstream": false},
    {"type": "im_wechatwork", "label": "企业微信 WeChatWork", "category": "im", "upstream": true, "downstream": false},
    {"type": "im_feishu", "label": "飞书 FeiShu", "category": "im", "upstream": true, "downstream": false},
    {"type": "ldap_ad", "label": "LDAP / Active Directory", "category": "ldap", "upstream": true, "downstream": true},
    {"type": "ldap_openldap", "label": "OpenLDAP", "category": "ldap", "upstream": true, "downstream": true},
    {"type": "db_mysql", "label": "MySQL", "category": "database", "upstream": true, "downstream": true},
    {"type": "db_postgresql", "label": "PostgreSQL", "category": "database", "upstream": true, "downstream": true},
    {"type": "db_sqlserver", "label": "SQL Server", "category": "database", "upstream": true, "downstream": true},
    {"type": "radius", "label": "RADIUS 服务器", "category": "radius", "upstream": true, "downstream": false},
    {"type": "http_api", "label": "HTTP API 认证", "category": "http", "upstream": true, "downstream": false}
  ]
}
```

1.10 10. 同步规则接口

1.10.1 10.1 获取上游同步规则列表

```
GET /api/open/sync/upstream/rules
```

1.10.2 10.2 创建上游同步规则

```
POST /api/open/sync/upstream/rules
```

请求体

```
{
  "name": "钉钉每日同步",
  "connectorId": 1,
  "schedule": "0 2 * * *",
  "enabled": true,
  "syncDepartments": true,
  "syncUsers": true,
  "conflictStrategy": "update"
}
```

1.10.3 10.3 触发同步

```
POST /api/open/sync/upstream/rules/:id/trigger
```

响应

```
{
  "success": true,
  "message": "同步任务已触发",
  "data": {
    "taskId": "sync-20260222-100000"
  }
}
```

1.10.4 10.4 获取字段映射

```
GET /api/open/sync/upstream/rules/:id/mappings
```

响应

```
{
  "success": true,
  "data": [
    {"sourceField": "name", "targetField": "displayName", "transform": ""},
    {"sourceField": "mobile", "targetField": "phone", "transform": ""},
    {"sourceField": "email", "targetField": "email", "transform": "lowercase"}
  ]
}
```

1.10.5 10.5 更新字段映射

```
PUT /api/open/sync/upstream/rules/:id/mappings
```

1.11 11. 通知渠道接口

1.11.1 11.1 获取通知渠道列表

```
GET /api/open/security/alerts/channels
```

响应

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "运维邮件",
      "type": "email",
      "enabled": true,
      "config": {"smtpHost": "smtp.example.com", "from": "alert@example.com"}
    },
    {
      "id": 2,
      "name": "钉钉机器人",
      "type": "dingtalk",
      "enabled": true,
      "config": {"webhook": "https://oapi.dingtalk.com/robot/..."}
    }
  ]
}
```

```
}
```

1.11.2 11.2 创建通知渠道

```
POST /api/open/security/alerts/channels
```

请求体（邮件）

```
{
  "name": "系统告警邮件",
  "type": "email",
  "enabled": true,
  "config": {
    "smtpHost": "smtp.example.com",
    "smtpPort": 465,
    "smtpUser": "alert@example.com",
    "smtpPassword": "xxx",
    "useTLS": true,
    "from": "alert@example.com",
    "to": ["admin@example.com"]
  }
}
```

请求体（钉钉机器人）

```
{
  "name": "钉钉告警群",
  "type": "dingtalk",
  "enabled": true,
  "config": {
    "webhook": "https://oapi.dingtalk.com/robot/send?access_token=xxx",
    "secret": "SECxxx"
  }
}
```

请求体（企业微信）

```
{
  "name": "企微告警群",
  "type": "wechatwork",
  "enabled": true,
  "config": {
    "webhook": "https://qyapi.weixin.qq.com/cgi-bin/webhook/send?key=xxx"
  }
}
```

请求体（飞书）

```
{
  "name": "飞书告警群",
  "type": "feishu",
  "enabled": true,
  "config": {
    "webhook": "https://open.feishu.cn/open-apis/bot/v2/hook/xxx",
    "secret": "xxx"
  }
}
```

请求体（短信-阿里云）

```
{
  "name": "阿里云短信",
  "type": "sms_aliyun",
  "enabled": true,
  "config": {
    "accessKeyId": "xxx",
    "accessKeySecret": "xxx",
    "signName": "Go-SyncFlow",
    "templateCode": "SMS_xxx"
  }
}
```

请求体（短信-云片）

```
{
  "name": "云片短信",
  "type": "sms_yunpian",
  "enabled": true,
  "config": {
    "apiKey": "xxx"
  }
}
```

1.11.3 11.3 测试通知渠道

```
POST /api/open/security/alerts/channels/:id/test
```

响应

```
{
  "success": true,
  "message": "测试消息发送成功"
}
```

1.11.4 11.4 更新通知渠道

```
PUT /api/open/security/alerts/channels/:id
```

1.11.5 11.5 删除通知渠道

```
DELETE /api/open/security/alerts/channels/:id
```

1.12 12. HTTPS 证书接口

1.12.1 12.1 获取 HTTPS 配置

```
GET /api/open/settings/https
```

响应

```
{
  "success": true,
  "data": {
    "enabled": true,
    "port": "8443",
    "certExists": true,
    "keyExists": true,
    "certExpiry": "2027-01-01T00:00:00Z",
    "certSubject": "CN=example.com",
    "domain": "example.com"
  }
}
```

1.12.2 12.2 更新 HTTPS 配置

```
PUT /api/open/settings/https
```

请求体

```
{
  "enabled": true,
  "port": "8443"
}
```

1.12.3 12.3 上传 SSL 证书

```
POST /api/open/settings/https/cert
```

请求 (multipart/form-data)

字段	类型	说明
cert	file	证书文件 (.cert/.pem)
key	file	私钥文件 (.key/.pem)

1.12.4 12.4 删除 SSL 证书

```
DELETE /api/open/settings/https/cert
```

1.13 13. 安全中心接口

1.13.1 13.1 获取安全仪表盘

```
GET /api/open/security/dashboard
```

响应

```
{
  "success": true,
  "data": {
    "securityScore": 85,
    "recentEvents": 12,
    "failedLogins24h": 5,
    "activeAlerts": 2,
    "lockedAccounts": 1,
    "blockedIPs": 3
  }
}
```

1.13.2 13.2 获取安全事件

```
GET /api/open/security/events
```

1.13.3 13.3 获取 IP 黑名单

```
GET /api/open/security/ip/blacklist
```

1.13.4 13.4 添加 IP 黑名单

```
POST /api/open/security/ip/blacklist
```

请求体

```
{
  "ip": "192.168.1.100",
  "reason": "暴力破解",
  "expireAt": "2026-03-01T00:00:00Z"
}
```

1.13.5 13.5 获取 IP 白名单

```
GET /api/open/security/ip/whitelist
```

1.13.6 13.6 添加 IP 白名单

```
POST /api/open/security/ip/whitelist
```

1.13.7 13.7 获取活跃会话

```
GET /api/open/security/sessions
```

1.13.8 13.8 终止会话

```
DELETE /api/open/security/sessions/:id
```

1.13.9 13.9 解锁账户

```
POST /api/open/security/lockouts/unlock-account
```

请求体

```
{"username": "zhangsan"}
```

1.14 14. 日志查询接口

1.14.1 14.1 登录日志 (Open API)

```
GET /api/open/logs/login
```

参数	类型	说明
page	int	页码
size	int	每页数量
username	string	用户名
status	string	success / failed
startDate	string	开始日期
endDate	string	结束日期

1.14.2 14.2 操作日志 (Open API)

```
GET /api/open/logs/operation
```

1.14.3 14.3 同步日志 (Open API)

```
GET /api/open/logs/sync
```

参数	类型	说明
direction	string	upstream / downstream
status	string	success / partial / failed
connectorId	int	连接器 ID

1.14.4 14.4 API 调用日志

```
GET /api/open/logs/api
```

参数	类型	说明
authType	string	apikey / jwt
method	string	GET / POST / PUT / DELETE
path	string	路径筛选
statusGroup	string	2xx / 4xx / 5xx

1.14.5 14.5 VPN 日志

```
GET /api/open/vpn/logs
```

1.15 15. 系统状态接口

1.15.1 15.1 获取系统状态 (Open API)

```
GET /api/open/system/status
```

响应

```
{
  "success": true,
  "data": {
    "version": "3.5.0",
    "uptime": "10d 5h 30m",
    "userCount": 1500,
    "activeUserCount": 1200,
    "groupCount": 50,
    "services": {
      "database": "connected",
      "ldap": "running",
      "vpn": "running",
      "scheduler": "running"
    },
    "lastSync": {
      "upstream": "2026-02-22T02:00:00Z",
      "downstream": "2026-02-22T02:05:00Z"
    }
  }
}
```

1.16 16. 触发同步接口

1.16.1 16.1 触发钉钉同步 (Open API)

```
POST /api/open/dingtalk/sync
```

1.16.2 16.2 获取同步状态 (Open API)

```
GET /api/open/dingtalk/sync/status
```

响应

```
{
  "success": true,
  "data": {
    "status": "running",
    "progress": 45,
    "currentStep": "同步用户数据",
    "startTime": "2026-02-22T10:00:00Z"
  }
}
```

1.16.3 16.3 触发全量同步

```
POST /api/open/sync/trigger-all
```

1.17 17. API 密钥管理接口

1.17.1 17.1 获取 API 密钥列表

```
GET /api/open/apikeys
```

1.17.2 17.2 创建 API 密钥

```
POST /api/open/apikeys
```

请求体

```
{
  "name": "第三方系统集成",
```

```
"permissions": ["read", "write"],
"ipWhitelist": ["192.168.1.0/24"],
"expireAt": "2027-01-01T00:00:00Z"
}
```

响应

```
{
  "success": true,
  "data": {
    "id": 1,
    "appId": "app_xxx",
    "appKey": "key_xxx_only_shown_once",
    "name": "第三方系统集成"
  }
}
```

1.17.3 17.3 重置 API 密钥

```
POST /api/open/apikeys/:id/reset
```

1.17.4 17.4 启用/禁用 API 密钥

```
PUT /api/open/apikeys/:id/toggle
```

1.17.5 17.5 删除 API 密钥

```
DELETE /api/open/apikeys/:id
```

1.18 18. 密码代理认证接口

1.18.1 18.1 获取密码代理配置

```
GET /api/open/settings/password-auth-proxy
```

响应

```
{
  "success": true,
  "data": {
```

```
"enabled": true,
"connectorId": 1,
"connectorType": "ldap_ad",
"learnPassword": true,
"updateSambaNT": true
}
}
```

1.18.2 18.2 更新密码代理配置

```
PUT /api/open/settings/password-auth-proxy
```

1.18.3 18.3 测试密码代理

```
POST /api/open/settings/password-auth-proxy/test
```

请求体

```
{
  "username": "testuser",
  "password": "testpass"
}
```

1.18.4 18.4 用户认证接口（OpenAPI 专用）

此接口用于通过 AppID/AppKey 调用，验证用户名密码是否正确，适用于第三方系统集成认证。

```
POST /api/open/auth/authenticate
```

请求头

Header	类型	必填	说明
X-App-ID	string	是	AppID
X-App-Key	string	是	AppKey（原始密钥）
Content-Type	string	是	application/json

请求体

```
{
  "username": "zhangsan",
  "password": "user_password"
```

```
}
```

注意：密码支持明文或 *SHA256* 哈希格式，系统会自动识别并验证。

成功响应

```
{
  "success": true,
  "code": "OK",
  "message": "认证成功",
  "data": {
    "userId": 1,
    "username": "zhangsan",
    "nickname": "张三",
    "email": "zhangsan@example.com",
    "phone": "13800138000",
    "status": 1,
    "groups": ["研发部", "管理层"],
    "roles": ["普通用户"],
    "lastLoginAt": "2026-02-25T10:30:00Z"
  }
}
```

失败响应

```
{
  "success": false,
  "code": "INVALID_PASSWORD",
  "message": "用户名或密码错误"
}
```

错误码说明

错误码	说明
OK	认证成功
INVALID_PARAMS	参数错误, 缺少 username 或 password
USER_NOT_FOUND	用户不存在
USER_DISABLED	用户已被禁用
USER_EXPIRED	账户已过期
USER_LOCKED	账户已被锁定
INVALID_PASSWORD	用户名或密码错误

特性说明

1. **支持密码代理**：如果本地密码验证失败且启用了密码代理，系统会自动尝试通过上游连接器验证
2. **密码学习**：通过密码代理认证成功后，密码会被学习到本地
3. **登录日志**：每次调用都会记录登录日志（类型为 OpenAPI）
4. **安全限制**：受账户锁定策略保护

调用示例（cURL）

```
#!/bin/bash
APP_ID="your_app_id"
APP_KEY="your_app_key"
BASE_URL="http://localhost:8080"

curl -X POST "${BASE_URL}/api/open/auth/authenticate" \
  -H "X-App-ID: ${APP_ID}" \
  -H "X-App-Key: ${APP_KEY}" \
  -H "Content-Type: application/json" \
  -d '{"username": "zhangsan", "password": "mypassword"}'
```

1.18.5 18.5 用户同步接口（含密码哈希）

重要说明：这是一个**特殊用途接口**，专门用于将用户数据（包括密码）同步到下游系统（如 LDAP、数据库、第三方应用）。

与普通用户接口的区别：
- `/api/open/users` - 用户管理接口，**不返回密码**，适用于一般用户查询
- `/api/open/sync/users` - 同步接口，**返回密码哈希**，仅用于系统集成同步

安全建议：请妥善保管调用此接口的 `AppID/AppKey`，避免密码哈希泄露。

获取用户列表（含密码哈希）

```
GET /api/open/sync/users
```

查询参数

参数	类型	必填	说明
page	int	否	页码，默认 1
size	int	否	每页数量，默认 50，最大 500
keyword	string	否	搜索关键词（用户名、昵称、邮箱、手机）

参数	类型	必填	说明
status	int	否	状态筛选：0= 禁用，1= 正常
group_id	int	否	用户组 ID 筛选
updated_after	string	否	增量同步：只返回此时间之后更新的用户（RFC3339 格式）

响应

```
{
  "success": true,
  "data": {
    "list": [
      {
        "id": 1,
        "username": "zhangsan",
        "nickname": "张三",
        "email": "zhangsan@example.com",
        "phone": "13800138000",
        "status": 1,
        "password_hash": "$2a$10$xxxxx...",
        "samba_nt_password": "A1B2C3D4E5F6...",
        "created_at": "2026-01-01T00:00:00Z",
        "updated_at": "2026-02-25T10:30:00Z",
        "last_login_at": "2026-02-25T08:00:00Z",
        "groups": ["研发部"],
        "roles": ["普通用户"]
      }
    ],
    "total": 100,
    "page": 1,
    "pageSize": 50
  },
  "message": "用户同步数据获取成功"
}
```

字段说明

字段	说明
password_hash	bcrypt 格式的密码哈希（可直接用于支持 bcrypt 的系统）

字段	说明
samba_nt_password	NT Hash 格式密码（用于 Samba/Active Directory/群晖 NAS）

获取单个用户详情（含密码哈希）

```
GET /api/open/sync/users/:id
```

响应结构同上

增量同步示例

获取最近24小时更新的用户

```
curl -X GET "${BASE_URL}/api/open/sync/users?updated_after=2026-02-24T00:00:00Z" \
-H "X-App-ID: ${APP_ID}" \
-H "X-App-Key: ${APP_KEY}"
```

使用场景

1. **下游系统同步**：将用户密码同步到其他系统（LDAP、数据库等）
2. **备份迁移**：导出用户数据进行系统迁移
3. **增量同步**：使用 `updated_after` 参数实现高效的增量同步

1.19 19. API 安全限制

1.19.1 19.1 禁止的操作

以下操作通过 API **无法执行**，以确保系统安全：

操作	说明
删除日志	所有日志（登录、操作、同步、API 调用、安全事件）只能通过系统定时清理任务删除，无法通过 API 删除
修改审计日志	审计日志为只读，无法修改或删除

1.19.2 19.2 日志清理机制

日志清理只能通过以下方式：

1. **自动清理**：根据「日志管理 → 日志设置」中配置的保留天数自动清理

2. 手动触发清理：管理员在界面点击「立即清理」按钮

配置项说明

日志类型	默认保留天数	说明
登录日志	90 天	用户登录/退出记录
操作日志	90 天	用户操作行为记录
同步日志	90 天	数据同步执行记录
API 调用日志	30 天	API 请求记录
安全事件	180 天	安全相关事件
VPN 日志	90 天	VPN 连接/活动记录

1.20 20. 接口汇总表

所有接口均支持 *AppID/AppKey* 签名认证，路径前缀统一为 */api/open*

1.20.1 认证接口

方法	路径	说明
POST	/api/open/auth/authenticate	验证用户名密码（OpenAPI 专用）

1.20.2 用户管理（不含密码）

以下接口用于日常用户管理，**不返回密码哈希**，确保安全。

方法	路径	说明
GET	/api/open/users	获取用户列表（ 不含密码 ）
POST	/api/open/users	创建用户
GET	/api/open/users/:id	获取用户详情
PUT	/api/open/users/:id	更新用户
DELETE	/api/open/users/:id	删除用户
PUT	/api/open/users/:id/status	更新用户状态
PUT	/api/open/users/:id/reset-password	重置密码

方法	路径	说明
POST	/api/open/users/batch-reset-password	批量重置密码
GET	/api/open/users/export	导出用户列表
GET	/api/open/groups	获取用户组列表
POST	/api/open/groups	创建用户组
PUT	/api/open/groups/:id	更新用户组
DELETE	/api/open/groups/:id	删除用户组
GET	/api/open/roles	获取角色列表
POST	/api/open/roles	创建角色
GET	/api/open/roles/:id	获取角色详情
PUT	/api/open/roles/:id	更新角色
DELETE	/api/open/roles/:id	删除角色
GET	/api/open/roles/:id/permissions	获取角色权限
PUT	/api/open/roles/:id/permissions	更新角色权限

1.20.3 用户同步（含密码哈希）

以下接口专门用于**下游系统同步**，返回用户密码哈希，请妥善保管 *AppID/AppKey*。

方法	路径	说明
GET	/api/open/sync/users	获取用户列表（含密码哈希，用于下游同步）
GET	/api/open/sync/users/:id	获取单个用户详情（含密码哈希）

1.20.4 单点登录 (SSO)

方法	路径	说明
GET	/api/open/sso/apps	获取 SSO 应用列表
POST	/api/open/sso/apps	创建 SSO 应用
GET	/api/open/sso/apps/:id	获取应用详情
PUT	/api/open/sso/apps/:id	更新 SSO 应用
DELETE	/api/open/sso/apps/:id	删除 SSO 应用

方法	路径	说明
GET	/api/open/sso/apps/:id/protocols	获取协议配置
PUT	/api/open/sso/apps/:id/protocols	更新协议配置
GET	/api/open/sso/apps/:id/mappings	获取属性映射
PUT	/api/open/sso/apps/:id/mappings	更新属性映射
GET	/api/open/sso/apps/:id/grants	获取授权用户
PUT	/api/open/sso/apps/:id/grants	更新授权用户
GET	/api/open/sso/logs	获取 SSO 审计日志
GET	/api/open/sso/integration/overview	获取集成概览

1.20.5 VPN 管理

方法	路径	说明
GET	/api/open/vpn/service/status	获取 VPN 服务状态
POST	/api/open/vpn/service/start	启动 VPN 服务
POST	/api/open/vpn/service/stop	停止 VPN 服务
POST	/api/open/vpn/service/restart	重启 VPN 服务
GET	/api/open/vpn/dashboard	获取 VPN 仪表盘
GET	/api/open/vpn/settings	获取 VPN 配置
PUT	/api/open/vpn/settings	更新 VPN 配置
GET	/api/open/vpn/groups	获取 VPN 用户组
POST	/api/open/vpn/groups	创建 VPN 用户组
PUT	/api/open/vpn/groups/:id	更新 VPN 用户组
DELETE	/api/open/vpn/groups/:id	删除 VPN 用户组
GET	/api/open/vpn/users	获取 VPN 用户
GET	/api/open/vpn/users/:id	获取 VPN 用户详情
PUT	/api/open/vpn/users/:id	更新 VPN 用户
POST	/api/open/vpn/users/:id/reset-pin	重置用户 PIN 码
GET	/api/open/vpn/online	获取在线用户
GET	/api/open/vpn/online/stats	获取在线统计
POST	/api/open/vpn/online/:token/kick	踢出用户
GET	/api/open/vpn/ip-maps	获取 IP 映射

方法	路径	说明
POST	/api/open/vpn/ip-maps	创建 IP 映射
PUT	/api/open/vpn/ip-maps/:id	更新 IP 映射
DELETE	/api/open/vpn/ip-maps/:id	删除 IP 映射
GET	/api/open/vpn/policies	获取访问策略
POST	/api/open/vpn/policies/set	设置策略
POST	/api/open/vpn/policies/del	删除策略
GET	/api/open/vpn/audit	获取 VPN 审计日志
GET	/api/open/vpn/user-logs	获取用户操作日志

1.20.6 上游同步

方法	路径	说明
GET	/api/open/sync/upstream/connectors	获取上游连接器列表
POST	/api/open/sync/upstream/connectors	创建上游连接器
GET	/api/open/sync/upstream/connectors/:id	获取连接器详情
PUT	/api/open/sync/upstream/connectors/:id	更新连接器
DELETE	/api/open/sync/upstream/connectors/:id	删除连接器
POST	/api/open/sync/upstream/connectors/:id/test	测试连接
GET	/api/open/sync/upstream/rules	获取上游同步规则
POST	/api/open/sync/upstream/rules	创建同步规则
PUT	/api/open/sync/upstream/rules/:id	更新同步规则
DELETE	/api/open/sync/upstream/rules/:id	删除同步规则
POST	/api/open/sync/upstream/rules/:id/trigger	触发同步
GET	/api/open/sync/upstream/rules/:id/mappings	获取字段映射
PUT	/api/open/sync/upstream/rules/:id/mappings	更新字段映射

1.20.7 下游同步

方法	路径	说明
GET	/api/open/sync/downstream/connectors	获取下游连接器列表
POST	/api/open/sync/downstream/connectors	创建下游连接器

方法	路径	说明
GET	/api/open/sync/downstream/connectors/:id	获取连接器详情
PUT	/api/open/sync/downstream/connectors/:id	更新连接器
DELETE	/api/open/sync/downstream/connectors/:id	删除连接器
POST	/api/open/sync/downstream/connectors/:id/test	测试连接
GET	/api/open/sync/downstream/rules	获取下游同步规则
POST	/api/open/sync/downstream/rules	创建同步规则
PUT	/api/open/sync/downstream/rules/:id	更新同步规则
DELETE	/api/open/sync/downstream/rules/:id	删除同步规则
POST	/api/open/sync/downstream/rules/:id/trigger	触发同步
GET	/api/open/sync/downstream/rules/:id/mappings	获取字段映射
PUT	/api/open/sync/downstream/rules/:id/mappings	更新字段映射
GET	/api/open/sync/connector-types	获取连接器类型
POST	/api/open/sync/trigger-all	触发全量同步

1.20.8 日志管理

方法	路径	说明
GET	/api/open/logs/system	获取系统日志
GET	/api/open/logs/system/stats	系统日志统计
GET	/api/open/logs/login	获取登录日志
GET	/api/open/logs/login/stats	登录日志统计
GET	/api/open/logs/operation	获取操作日志
GET	/api/open/logs/operation/stats	操作日志统计
GET	/api/open/logs/sync	获取同步日志
GET	/api/open/logs/api	获取 API 访问日志
GET	/api/open/logs/api/stats	API 日志统计
GET	/api/open/settings/log-retention	获取日志保留策略
PUT	/api/open/settings/log-retention	更新日志保留策略
POST	/api/open/settings/log-retention/clean	立即清理日志

1.20.9 通知渠道

方法	路径	说明
GET	/api/open/security/alerts/channels	获取通知渠道
POST	/api/open/security/alerts/channels	创建通知渠道
PUT	/api/open/security/alerts/channels/:id	更新通知渠道
DELETE	/api/open/security/alerts/channels/:id	删除通知渠道
POST	/api/open/security/alerts/channels/:id/test	测试通知渠道
GET	/api/open/security/alerts/rules	获取告警规则
POST	/api/open/security/alerts/rules	创建告警规则
PUT	/api/open/security/alerts/rules/:id	更新告警规则
DELETE	/api/open/security/alerts/rules/:id	删除告警规则

1.20.10 消息模板与策略

方法	路径	说明
GET	/api/open/notify/templates	获取消息模板
POST	/api/open/notify/templates	创建消息模板
PUT	/api/open/notify/templates/:id	更新消息模板
DELETE	/api/open/notify/templates/:id	删除消息模板
GET	/api/open/notify/policies	获取消息策略
POST	/api/open/notify/policies	创建/更新策略

1.20.11 安全中心

方法	路径	说明
GET	/api/open/security/dashboard	获取安全仪表盘
GET	/api/open/security/events	获取安全事件
PUT	/api/open/security/events/:id/resolve	处理安全事件
GET	/api/open/security/login-attempts	获取登录尝试记录
GET	/api/open/security/lockouts	获取锁定记录
POST	/api/open/security/lockouts/unlock-account	解锁账户
POST	/api/open/security/lockouts/unlock-ip	解锁 IP
GET	/api/open/security/ip/blacklist	获取 IP 黑名单

方法	路径	说明
POST	/api/open/security/ip/blacklist	添加 IP 黑名单
DELETE	/api/open/security/ip/blacklist/:id	移除 IP 黑名单
GET	/api/open/security/ip/whitelist	获取 IP 白名单
POST	/api/open/security/ip/whitelist	添加 IP 白名单
DELETE	/api/open/security/ip/whitelist/:id	移除 IP 白名单
GET	/api/open/security/sessions	获取所有会话
DELETE	/api/open/security/sessions/:id	终止会话
DELETE	/api/open/security/sessions/user/:userId	终止用户所有会话
GET	/api/open/security/config/:key	获取安全配置
PUT	/api/open/security/config/:key	更新安全配置

1.20.12 系统设置

方法	路径	说明
GET	/api/open/settings/ui	获取 UI 配置
PUT	/api/open/settings/ui	更新 UI 配置
GET	/api/open/settings/ldap	获取 LDAP 配置
PUT	/api/open/settings/ldap	更新 LDAP 配置
POST	/api/open/settings/ldap/test	测试 LDAP 连接
GET	/api/open/settings/https	获取 HTTPS 配置
PUT	/api/open/settings/https	更新 HTTPS 配置
POST	/api/open/settings/https/cert	上传 SSL 证书
DELETE	/api/open/settings/https/cert	删除 SSL 证书
GET	/api/open/settings/2fa	获取 2FA 配置
PUT	/api/open/settings/2fa	更新 2FA 配置
GET	/api/open/system/status	获取系统状态

1.20.13 API 密钥管理

方法	路径	说明
GET	/api/open/apikeys	获取 API 密钥列表

方法	路径	说明
POST	/api/open/apikeys	创建 API 密钥
GET	/api/open/apikeys/:id	获取密钥详情
PUT	/api/open/apikeys/:id	更新 API 密钥
POST	/api/open/apikeys/:id/reset	重置密钥
PUT	/api/open/apikeys/:id/toggle	切换状态
DELETE	/api/open/apikeys/:id	删除密钥

1.20.14 密码代理认证

方法	路径	说明
GET	/api/open/settings/password-auth-proxy	获取配置
PUT	/api/open/settings/password-auth-proxy	更新配置
POST	/api/open/settings/password-auth-proxy/test	测试认证

1.21 20. 错误码参考

错误码	HTTP 状态	说明
INVALID_SIGNATURE	401	签名验证失败
TIMESTAMP_EXPIRED	401	时间戳过期
INVALID_APP_ID	401	AppID 不存在
APP_KEY_DISABLED	403	API 密钥已禁用
PERMISSION_DENIED	403	无权限
NOT_FOUND	404	资源不存在
VALIDATION_ERROR	400	参数验证失败
DUPLICATE_ENTRY	409	数据重复
INTERNAL_ERROR	500	服务器内部错误

1.22 21. 安全说明

1.22.1 API 安全限制

- **禁止操作管理员账户：**不允许通过 API 修改、删除 admin 账户
- **禁止分配超级管理员角色：**API 无法给用户分配超级管理员权限

1.22.2 频率限制

Open API **不设默认频率限制**。如需限制特定 API 密钥的调用频率，请在创建/编辑 API 密钥时设置。